

LOAD BALANCER LAYER 4 VS LAYER 7

DOSEN PENGAMPU: RUNAL REZKIAWAN, S.KOM.,M.T



Disusun Oleh:

NADILLA GETALIA

105841106623

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan "**Load Balancer Layer 4 vs Layer 7**" sebagai salah satu tugas pada mata kuliah **Scalable Systems Design**.

Laporan ini disusun untuk memahami konsep load balancing serta implementasi Load Balancer Layer 4 menggunakan HAProxy dan Load Balancer Layer 7 menggunakan Nginx dalam lingkungan Docker. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca.

Makassar, 11 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR DAN TABEL	iv
BAB I PENDAHULUAN.....	1
A. Latar Belakang	1
B. Rumusan Masalah	1
C. Tujuan.....	2
D. Manfaat	2
BAB II IMPLEMENTASI SISTEM	3
A. Arsitektur Sistem.....	3
B. Struktur Folder Proyek.....	3
C. Implementasi Load Balancer Layer 7 Menggunakan Nginx	4
D. Implementasi Load Balancer Layer 4 Menggunakan HAProxy	4
E. Implementasi Docker Compose	5
F. Menjalankan Sistem	5
BAB III HASIL DAN PEMBAHASAN.....	6
A. Hasil Implementasi Sistem.....	6
B. Pengujian Load Balancer Layer 7 Menggunakan Nginx	7
C. Pengujian Load Balancer Layer 4 Menggunakan HAProxy	8
D. Monitoring Resource Container	8
E. Perbandingan Load Balancer Layer 4 dan Layer 7	9
BAB IV PENUTUP	9
A. Kesimpulan	9
B. Saran.....	10

DAFTAR GAMBAR DAN TABEL

Gambar 2. 1 Arsitektur Sistem	3
Gambar 2. 2 Struktur Folder Proyek	4
Gambar 3. 1 Hasil menjalankan Docker Compose	6
Gambar 3. 2 Hasil Perintah docker ps.....	7
Gambar 3. 3 Pengujian Load Balancer Layer 7 Menggunakan Nginx	7
Gambar 3. 4 Pengujian Load Balancer Layer 4 menggunakan HAProxy	8
Gambar 3. 5 Monitoring Container Menggunakan docker stats	8
Tabel 3. 1 Perbandingan Layer 4 dan Layer 7	9

BAB I PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi telah menyebabkan peningkatan jumlah pengguna dan lalu lintas data pada berbagai aplikasi berbasis web. Kondisi ini menuntut sistem untuk mampu menangani banyak permintaan secara bersamaan tanpa mengalami penurunan kinerja. Salah satu solusi yang digunakan untuk mengatasi permasalahan tersebut adalah penerapan load balancer.

Load balancer merupakan mekanisme yang berfungsi untuk mendistribusikan lalu lintas jaringan atau permintaan pengguna ke beberapa *server backend* sehingga beban kerja tidak terpusat pada satu *server* saja. Dengan adanya *load balancing*, sistem dapat meningkatkan ketersediaan layanan, performa, dan keandalan dalam melayani pengguna.

Berdasarkan model *OSI*, *load balancer* dapat bekerja pada berbagai lapisan, di antaranya *Layer 4 (Transport Layer)* dan *Layer 7 (Application Layer)*. *Load Balancer Layer 4* melakukan distribusi trafik berdasarkan informasi jaringan seperti alamat IP dan nomor *port*, sedangkan *Load Balancer Layer 7* mampu melakukan distribusi berdasarkan informasi aplikasi seperti URL, header HTTP, dan jenis permintaan yang diterima. Kedua jenis *load balancer* tersebut memiliki karakteristik, kelebihan, dan kekurangan yang berbeda.

Pada praktikum ini dilakukan implementasi *Load Balancer Layer 4* menggunakan HAProxy dan *Load Balancer Layer 7* menggunakan Nginx yang dijalankan dalam lingkungan Docker. Melalui implementasi tersebut, diharapkan dapat dipahami perbedaan cara kerja, konfigurasi, serta karakteristik masing-masing *load balancer* dalam mendistribusikan permintaan ke beberapa *backend server*.

B. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam matakuliah ini adalah sebagai berikut:

1. Bagaimana cara mengimplementasikan *Load Balancer Layer 4* menggunakan HAProxy pada lingkungan *Docker*?

2. Bagaimana cara mengimplementasikan *Load Balancer Layer 7* menggunakan Nginx pada lingkungan Docker?
3. Bagaimana mekanisme distribusi trafik pada *Load Balancer Layer 4* dan *Layer 7*?
4. Apa perbedaan karakteristik antara *Load Balancer Layer 4* dan *Layer 7* dalam menangani permintaan pengguna?

C. Tujuan

Tujuan yang ingin dicapai dalam matakuliah ini adalah:

1. Mengimplementasikan *Load Balancer Layer 4* menggunakan HAProxy.
2. Mengimplementasikan *Load Balancer Layer 7* menggunakan Nginx.
3. Membangun beberapa backend server menggunakan Docker sebagai target distribusi trafik.
4. Menganalisis perbedaan cara kerja *Load Balancer Layer 4* dan *Layer 7*.
5. Memahami proses distribusi beban ke beberapa server backend menggunakan teknik load balancing.

D. Manfaat

Manfaat yang diperoleh dari pelaksanaan praktikum ini adalah sebagai berikut:

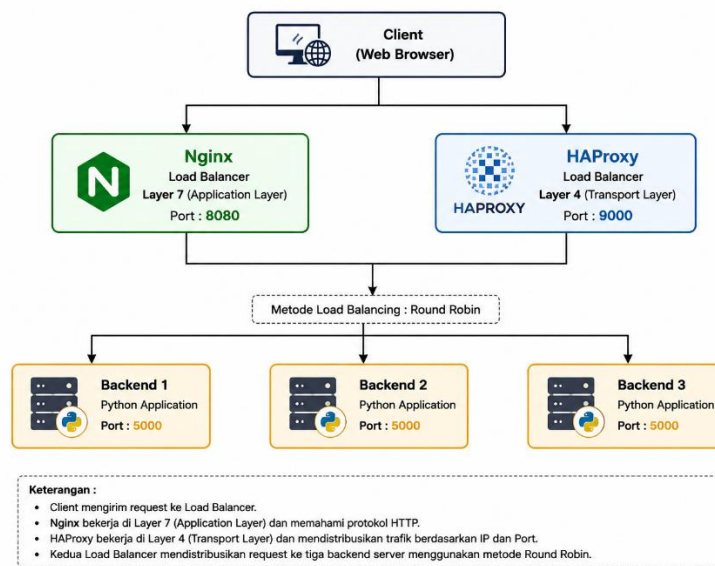
1. Menambah pemahaman mengenai konsep *load balancing* dalam sistem terdistribusi.
2. Mengetahui perbedaan fungsi dan karakteristik *Load Balancer Layer 4* dan *Layer 7*.
3. Meningkatkan kemampuan dalam menggunakan Docker untuk membangun dan mengelola layanan berbasis container.
4. Memahami penggunaan HAProxy dan Nginx sebagai solusi *load balancing*.
5. Menjadi dasar dalam pengembangan sistem yang memiliki kebutuhan skalabilitas dan ketersediaan layanan yang tinggi.

BAB II IMPLEMENTASI SISTEM

A. Arsitektur Sistem

Pada implementasi ini digunakan dua jenis *load balancer*, yaitu *Load Balancer Layer 4* menggunakan HAProxy dan *Load Balancer Layer 7* menggunakan Nginx. Kedua *load balancer* tersebut digunakan untuk mendistribusikan permintaan pengguna ke tiga *backend server* yang berjalan dalam *container Docker*.

Arsitektur sistem yang digunakan dapat digambarkan sebagai berikut:

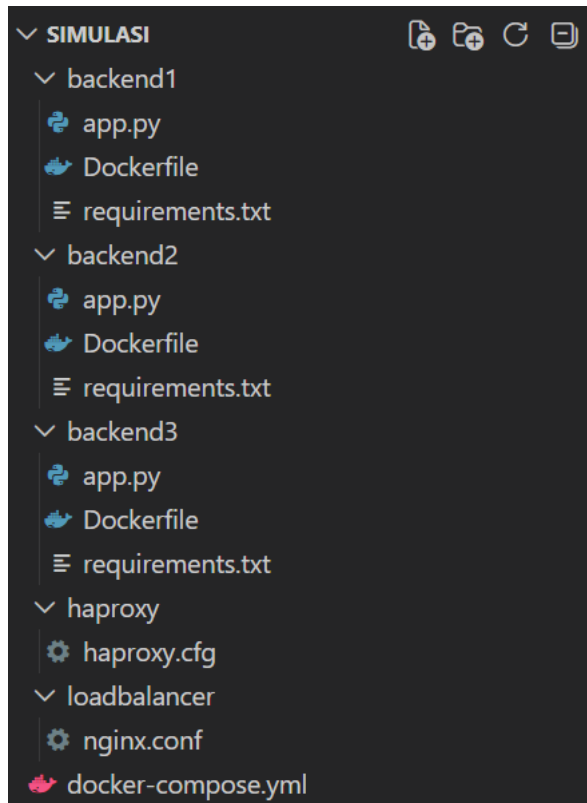


Gambar 2. 1 Arsitektur Sistem

Pada arsitektur tersebut, pengguna dapat mengakses layanan melalui Nginx pada port 8080 atau melalui HAProxy pada port 9000. Selanjutnya, request akan didistribusikan ke *backend server* menggunakan metode Round Robin.

B. Struktur Folder Proyek

Struktur folder proyek yang digunakan adalah sebagai berikut:



Gambar 2. 2 Struktur Folder Proyek

Setiap folder backend berisi aplikasi Python yang berfungsi sebagai server tujuan. Folder *loadbalancer* berisi konfigurasi Nginx, sedangkan folder haproxy berisi konfigurasi HAProxy. Seluruh layanan dijalankan secara bersamaan menggunakan Docker Compose.

C. Implementasi Load Balancer Layer 7 Menggunakan Nginx

Nginx digunakan sebagai *Load Balancer Layer 7* yang bekerja pada lapisan aplikasi (Application Layer). Nginx dapat memahami protokol HTTP sehingga mampu melakukan distribusi request berdasarkan informasi aplikasi. Konfigurasi Nginx disimpan pada file `nginx.conf`. Pada konfigurasi tersebut ditentukan tiga *backend server* yang menjadi tujuan distribusi trafik.

Nginx dijalankan menggunakan container Docker dan dipublikasikan pada port 8080 sehingga dapat diakses melalui browser menggunakan alamat:

<http://localhost:8080>

D. Implementasi Load Balancer Layer 4 Menggunakan HAProxy

HAProxy digunakan sebagai Load Balancer Layer 4 yang bekerja pada lapisan transport (Transport Layer). HAProxy melakukan distribusi trafik

berdasarkan alamat IP dan nomor port tanpa memperhatikan isi data aplikasi. Konfigurasi HAProxy disimpan pada file haproxy.cfg. Pada konfigurasi tersebut ditentukan tiga backend server yang menerima trafik dari pengguna.

HAProxy dijalankan menggunakan container Docker dan dipublikasikan pada port 9000 sehingga dapat diakses menggunakan alamat:

<http://localhost:9000>

Metode *load balancing* yang digunakan adalah *Round Robin*, yaitu mendistribusikan request secara bergantian ke setiap *backend server* yang tersedia.

E. Implementasi Docker Compose

Docker Compose digunakan untuk mengelola dan menjalankan seluruh container secara bersamaan. Dalam implementasi ini terdapat lima layanan utama, yaitu:

1. Backend Server 1
2. Backend Server 2
3. Backend Server 3
4. Nginx sebagai Load Balancer Layer 7
5. HAProxy sebagai Load Balancer Layer 4

Seluruh layanan dijalankan menggunakan satu file konfigurasi docker-compose.yml sehingga proses deployment menjadi lebih mudah dan terstruktur.

F. Menjalankan Sistem

Untuk menjalankan seluruh layanan digunakan perintah berikut:

```
docker compose up -d --build
```

Perintah tersebut akan membangun image yang diperlukan dan menjalankan seluruh container secara otomatis.

Untuk memastikan seluruh container berjalan dengan baik digunakan perintah:

```
docker ps
```

Hasil pengujian menunjukkan bahwa *container backend1*, *backend2*, *backend3*, Nginx, dan HAProxy berhasil berjalan sesuai konfigurasi yang telah dibuat.

BAB III HASIL DAN PEMBAHASAN

A. Hasil Implementasi Sistem

Setelah seluruh konfigurasi selesai dibuat, sistem dijalankan menggunakan *Docker Compose* dengan perintah:

```
docker compose up -d --build
```

Perintah tersebut berhasil membangun dan menjalankan seluruh container yang terdiri atas tiga backend server, satu container Nginx sebagai *Load Balancer Layer 7*, dan satu container HAProxy sebagai *Load Balancer Layer 4*.

```
C:\Users\ASUS\Downloads\simulasi\simulasi>docker compose up -d --build
[+] Building 3.5s (29/29) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.55kB 0.0s
=> [backend1 internal] load build definition from Docker 0.0s
=> => transferring dockerfile: 177B 0.0s
=> [backend3 internal] load build definition from Docker 0.0s
=> => transferring dockerfile: 177B 0.0s
=> [backend2 internal] load build definition from Docker 0.0s
=> => transferring dockerfile: 177B 0.0s
=> [backend1 internal] load metadata for docker.io/libra 2.7s
[auth] library/python:pull token for registry-1.docker 0.0s
=> [backend2 internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend3 internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend1 internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend3 1/5] FROM docker.io/library/python:3.11@sha 0.0s
=> => resolve docker.io/library/python:3.11@sha256:58a54 0.0s
=> [backend1 internal] load build context 0.0s
=> => transferring context: 93B 0.0s
=> [backend3 internal] load build context 0.0s
=> => transferring context: 93B 0.0s
=> [backend2 internal] load build context 0.0s
=> => transferring context: 93B 0.0s
=> CACHED [backend3 2/5] WORKDIR /app 0.0s
=> CACHED [backend1 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend1 4/5] RUN pip install -r requirements 0.0s
=> CACHED [backend1 5/5] COPY . . 0.0s
=> CACHED [backend2 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend2 4/5] RUN pip install -r requirements 0.0s
=> CACHED [backend2 5/5] COPY . . 0.0s
=> CACHED [backend3 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend3 4/5] RUN pip install -r requirements 0.0s
=> CACHED [backend3 5/5] COPY . . 0.0s
=> [backend1] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:5fac0f8d9efe9ddc1f9c100c 0.0s
=> => exporting config sha256:b4e920497543dc13418662ffe5 0.0s
=> => exporting attestation manifest sha256:bfe7a804ddba 0.1s
=> => exporting manifest list sha256:9e4084aa30f86e759d0 0.0s
=> => naming to docker.io/library/simulasi-backend1:late 0.0s
=> => unpacking to docker.io/library/simulasi-backend1:l 0.0s
=> [backend2] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:a0b0832e99fdb503fbc6e8ef 0.0s
=> => exporting config sha256:dd0599abcf9790960b3ad29c62 0.0s
=> => exporting attestation manifest sha256:fde80198f4e3 0.1s
=> => exporting manifest list sha256:052498b4ad9da97a142 0.0s
=> => naming to docker.io/library/simulasi-backend2:late 0.0s
=> => unpacking to docker.io/library/simulasi-backend2:l 0.0s
=> [backend3] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:2e37aea71c977c8a2c641eb4 0.0s
=> => exporting config sha256:41523321396b9cd8566ec771eb 0.0s
=> => exporting attestation manifest sha256:b5e946253ae3 0.1s
=> => exporting manifest list sha256:c6f88cc30bcd722fc97 0.0s
=> => naming to docker.io/library/simulasi-backend3:late 0.0s
=> => unpacking to docker.io/library/simulasi-backend3:l 0.0s
=> [backend1] resolving provenance for metadata file 0.1s
=> [backend2] resolving provenance for metadata file 0.0s
=> [backend3] resolving provenance for metadata file 0.0s
[+] Running 9/9
✓ simulasi-backend3 Built 0.0s
✓ simulasi-backend1 Built 0.0s
✓ simulasi-backend2 Built 0.0s
✓ Network simulasi-default Created 0.1s
✓ Container simulasi-backend2-1 Started 1.0s
✓ Container simulasi-backend3-1 Started 0.9s
✓ Container simulasi-backend1-1 Started 1.0s
✓ Container simulasi-haproxy-1 Started 1.3s
✓ Container simulasi-loadbalancer-1 Started 1.2s
```

Gambar 3. 1 Hasil menjalankan Docker Compose

Untuk memastikan seluruh layanan berjalan dengan baik digunakan perintah:

docker ps

Berdasarkan hasil pengujian, seluruh container berada pada status **Up**, yang menunjukkan bahwa sistem berhasil dijalankan sesuai dengan konfigurasi yang telah dibuat.

```
C:\Users\ASUS\Downloads\simulasi\simulasi>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
2830408b170c	haproxy:latest	"docker-entrypoint.s..."	17 seconds ago	Up 16 seconds	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
000/tcp	simulasi-haproxy-1	"/docker-entrypoint..."	17 seconds ago	Up 16 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
a44e7e862135	nginx:latest	"python app.py"	17 seconds ago	Up 16 seconds	
tcp	simulasi-loadbalancer-1	"python app.py"	17 seconds ago	Up 16 seconds	
0223d2ff5905	simulasi-backend1	"python app.py"	17 seconds ago	Up 16 seconds	
268904b46ded	simulasi-backend3	"python app.py"	17 seconds ago	Up 16 seconds	
60ae8d62ab10	simulasi-backend3-1	"python app.py"	17 seconds ago	Up 16 seconds	
59bef63b66bc	simulasi-backend2	"python app.py"	17 seconds ago	Up 16 seconds	
28fa9ca746b7	simulasi-backend2-1	"python app.py"	17 seconds ago	Up 16 seconds	
k8s_nodejs-writer_nodejs-writer_default_b97e6548-bc1a-4573-8c61-92783b330d3b_3		"docker-entrypoint.s..."	55 minutes ago	Up 55 minutes	
e1017c9a1441	94ef2bdc0b32	"/docker-entrypoint..."	55 minutes ago	Up 55 minutes	
k8s_nginx_nginx_default_79c65ac9-7d92-44be-a843-df62190b68da_3					

Gambar 3. 2 Hasil Perintah docker ps

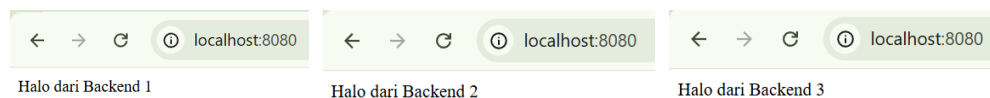
B. Pengujian Load Balancer Layer 7 Menggunakan Nginx

Pengujian *Load Balancer Layer 7* dilakukan dengan mengakses alamat:

<http://localhost:8080>

Nginx bertindak sebagai *load balancer* yang bekerja pada lapisan aplikasi (*Application Layer*). Pada implementasi ini Nginx menggunakan metode Round Robin untuk mendistribusikan request ke tiga *backend server* yang tersedia.

Hasil pengujian menunjukkan bahwa request dari pengguna berhasil diteruskan ke backend server secara bergantian. Hal ini membuktikan bahwa Nginx berhasil menjalankan fungsi *load balancing* pada *Layer 7*.



Gambar 3. 3 Pengujian *Load Balancer Layer 7* Menggunakan Nginx

Pada *Layer 7*, Nginx dapat memahami protokol HTTP sehingga mampu melakukan pengaturan trafik berdasarkan informasi aplikasi seperti URL, header HTTP, maupun jenis request yang diterima. Kemampuan tersebut membuat Nginx lebih fleksibel dalam menangani berbagai kebutuhan aplikasi web modern.

C. Pengujian Load Balancer Layer 4 Menggunakan HAProxy

Pengujian *Load Balancer Layer 4* dilakukan dengan mengakses alamat:

<http://localhost:9000>

HAProxy berfungsi sebagai load balancer yang bekerja pada lapisan transport (*Transport Layer*). Pada implementasi ini HAProxy menggunakan metode Round Robin untuk mendistribusikan koneksi ke tiga *backend server* yang tersedia.

Hasil pengujian menunjukkan bahwa request berhasil diteruskan ke *backend server* secara bergantian sesuai dengan konfigurasi yang telah dibuat.



Gambar 3. 4 Pengujian *Load Balancer Layer 4* menggunakan HAProxy

Pada *Layer 4*, HAProxy melakukan distribusi trafik berdasarkan alamat IP dan nomor port tanpa memperhatikan isi dari paket data yang dikirimkan. Pendekatan ini membuat proses *load balancing* menjadi lebih cepat karena tidak memerlukan pemeriksaan terhadap data aplikasi.

D. Monitoring Resource Container

Monitoring dilakukan menggunakan perintah:

`docker stats`

Perintah tersebut digunakan untuk melihat penggunaan sumber daya oleh setiap container yang sedang berjalan, seperti penggunaan CPU, memori, serta aktivitas jaringan.

C:\Windows\System32\cmd.exeWindows PowerShell

CONTAINER ID	NAME	BLOCK I/O	PIDS	CPU %	MEM USAGE / LIMIT	MEM %	
59bef63b66bc	ks_nodejs-writer_nodejs-writer_default_b97e6548-bc1a-4573-8c61-92783b330d3b_3	0B / 0B	33.4MB / 6.87MB	11	0.00%	6.816MiB / 7.459GiB	0.09%
e1017c9a1441	ks_nginx_nginx_default_79c65ac9-7d92-44be-a843-df62198b68da_3	0B / 0B	0B / 12.3kB	17	0.00%	13.19MiB / 7.459GiB	0.17%
268904b46ded	simulasi-backend3-1	298kB / 158kB	0B / 147kB	1	0.02%	21.27MiB / 7.459GiB	0.28%
0223d2ff5905	simulasi-backend1-1	299kB / 159kB	0B / 147kB	1	0.02%	21.27MiB / 7.459GiB	0.28%
60ae8d62ab10	simulasi-backend2-1	297kB / 158kB	0B / 147kB	1	0.02%	21.31MiB / 7.459GiB	0.28%
a44e7e862135	simulasi-loadbalancer-1	8.68kB / 6.88kB	0B / 12.3kB	2	0.00%	2.516MiB / 7.459GiB	0.03%
2830408b170c	simulasi-haproxy-1	484kB / 888kB	0B / 0B	17	0.06%	88.59MiB / 7.459GiB	1.16%

Gambar 3. 5 Monitoring Container Menggunakan docker stats

Berdasarkan hasil monitoring, seluruh container berhasil berjalan dengan penggunaan sumber daya yang relatif rendah. Container *backend1*, *backend2*, dan *backend3* hanya menggunakan sebagian kecil CPU dan memori karena

aplikasi yang dijalankan masih sederhana. *Container Nginx* dan *HAProxy* juga menunjukkan penggunaan *resource* yang rendah sehingga dapat disimpulkan bahwa mekanisme *load balancing* yang diimplementasikan tidak memberikan beban yang signifikan terhadap sistem.

E. Perbandingan Load Balancer Layer 4 dan Layer 7

Berdasarkan hasil implementasi dan pengujian, terdapat beberapa perbedaan antara *Load Balancer Layer 4* dan *Layer 7* sebagaimana ditunjukkan pada Tabel 3.1.

Tabel 3. 1 Perbandingan Layer 4 dan Layer 7

Aspek	Layer 4 (HAProxy)	Layer 7 (Nginx)
Lapisan OSI	Transport Layer	Application Layer
Dasar Distribusi	IP Address dan Port	HTTP, URL, Header
Kecepatan	Lebih Cepat	Seidkit Lebih Lambat
Fleksibilitas	Terbatas	Lebih Fleksibel
Software	HAProxy	Nginx
Port Pengujian	9000	8080

Berdasarkan pengujian yang dilakukan, kedua load balancer berhasil mendistribusikan request ke tiga backend server yang tersedia. HAProxy pada *Layer 4* menunjukkan proses distribusi yang lebih sederhana karena hanya memperhatikan informasi jaringan berupa alamat IP dan port. Sementara itu, Nginx pada *Layer 7* memiliki kemampuan yang lebih kompleks karena dapat memahami protokol HTTP dan melakukan pengelolaan trafik berdasarkan informasi aplikasi. Dalam implementasi ini, *Layer 4* lebih cocok digunakan pada lingkungan yang membutuhkan performa tinggi dan latensi rendah. Sebaliknya, *Layer 7* lebih sesuai digunakan pada aplikasi web yang memerlukan pengaturan trafik yang lebih spesifik dan fleksibel.

BAB IV PENUTUP

A. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem load balancing menggunakan Docker berhasil dijalankan dengan memanfaatkan tiga backend server, satu *Load Balancer Layer 7* menggunakan Nginx, dan satu *Load Balancer Layer 4* menggunakan HAProxy. Pengujian menunjukkan bahwa kedua *load balancer* mampu

mendistribusikan request ke *backend server* yang tersedia menggunakan metode *Round Robin*.

Load Balancer Layer 4 yang diimplementasikan menggunakan HAProxy bekerja pada lapisan transport dengan mendistribusikan trafik berdasarkan alamat IP dan nomor port. Pendekatan ini memberikan performa yang lebih cepat karena tidak memproses isi paket data. Sementara itu, *Load Balancer Layer 7* yang diimplementasikan menggunakan Nginx bekerja pada lapisan aplikasi dan mampu memahami protokol HTTP sehingga lebih fleksibel dalam mengelola trafik berdasarkan kebutuhan aplikasi.

Dari hasil perbandingan dapat disimpulkan bahwa *Layer 4* lebih sesuai digunakan pada sistem yang membutuhkan kecepatan dan efisiensi tinggi, sedangkan *Layer 7* lebih cocok digunakan pada aplikasi web yang memerlukan pengaturan trafik yang lebih kompleks dan spesifik. Dengan demikian, pemilihan jenis load balancer harus disesuaikan dengan kebutuhan sistem yang akan dibangun.

B. Saran

Berdasarkan implementasi yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan lebih lanjut, yaitu:

1. Menambahkan jumlah *backend server* untuk menguji kemampuan load balancer dalam menangani skala sistem yang lebih besar.
2. Melakukan pengujian performa menggunakan tools seperti Apache Benchmark (AB), JMeter, atau Locust untuk memperoleh data kuantitatif mengenai kinerja *Layer 4* dan *Layer 7*.
3. Menerapkan algoritma load balancing lain, seperti Least Connection atau Weighted Round Robin, untuk membandingkan efektivitas distribusi trafik.
4. Menambahkan mekanisme monitoring dan logging agar kondisi server dapat dipantau secara real-time.
5. Mengimplementasikan sistem pada lingkungan cloud atau server produksi sehingga dapat digunakan pada skenario yang lebih mendekati kondisi nyata.